Examen Hibernate amb DAO

0. CONTROL DE CANVIS (Obligatori)

	Realitzat per	Revisat per	Aprovat per
Nom i cognom (últim canvi)	**Clàudia Reina**		
Càrrec (últim canvi)	Professora		
Data (últim canvi)	2026-10-07		

Llistat de les modificacions (S'inicia i acaba a la mateixa reunió només posar la revisió 0.0 per saber la data de creació del document)		
Revisió núm	**Data**	**Descripció de la modificació**
0.0	2026-06-02	Creació del document
0.1	2026-10-07	Optimització enunciat
0.2		
0.3		
0.4		

★ ***Tan bon punt el document s'imprimeixi o es descarregui perd la seva vigència.***

Enunciat de l'examen

Has de dissenyar un model de dades amb Hibernate per a un sistema de gestió de brawlers i modes de joc en Brawl Stars. Implementar el patró DAO per a l'entitat principal i, si cal, per secundàries.
L'examen es divideix en dues parts: creació del model de dades i implementació del patró DAO.

Heu de crear un projecte Maven amb GroupId el vostre nom i cognoms i ArtifactId Hib{1rCognom}{Nom}.

El projecte haurà de tenir la següent estructura:

```
Hib{1rCognom}{Nom}/
│
├── pom.xml                    # Configuració Maven amb dependències
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── modelDades/        # Entitats Hibernate
│   │   │   │   ├── …
│   │   │   │   └── …
│   │   │   │
│   │   │   └── patroDao/          # Implementació del patró DAO
│   │   │       ├── …
│   │   │       └── Main.java            # Classe principal d'execució
│   │   │
│   │   └── resources/
│   │       └── hibernate.cfg.xml      # Configuració Hibernate
│   │
└── test/…
```

Part 1: Model de dades.

Crea un model de dades amb Hibernate per gestionar brawlers i partides dins de Brawl Stars. Totes les classes han de tenir hashCode i equals per id, a més d'un toString. Les entitats principals són:

1. **Brawler**: Representa un brawler jugable amb els següents atributs:
    a. Identificador autoincrementable
    b. nom (únic)
    c. rang (enum: BRONZE, PLATA, OR, DIAMANT). Per defecte inicia a BRONZE.
    d. trofeus (int, no pot ser negatiu, comença a 0).
    e. vida (double, de 0 a 10, no pot ser negatiu ni superar 10, comença a 10).
    f. El constructor només ha de demanar nom
    g. Setters i Getters:
        i. Gestiona des de Brawler la inserció o esborrat a les altres classes.
        ii. Assegura't de controlar quan no s'ha d'inserir un element si ja existeix.
        iii. Quan s'assigna una Partida activa a un Brawler, aplica el seu cost de vida. Assegura't que la vida no baixi de zero.
2. **Classe**: Defineix el tipus de brawler. Atributs:
    a. Identificador autoincrementable
    b. nom (únic)
    c. habilitat (nom de l'habilitat especial)
    d. dany (double, dany de l'habilitat)
    e. Constructor complet (excepte id)
3. **Partida**: Classe abstracta que representa una partida jugada en un mode específic. Atributs:
    a. Identificador autoincrementable
    b. titol
    c. descripcio
    d. recompensaTrofeus (int, trofeus a guanyar)
    e. costVida (double, vida consumida en partida)
    f. Constructor complet (excepte id)
4. **PartidaRanked** i **PartidaAmistosa**: Subclasses de Partida.
    a. No tenen atributs propis: només el constructor, que crida al de Partida.
    b. L'herència s'ha de mapejar en una única taula (InheritanceType.SINGLE_TABLE) amb una columna discriminadora anomenada tipus.
    c. No afegeixis cap atribut «tipus» a Partida: el tipus de cada partida es distingeix per la classe de l'objecte.

Requisits del model:

- Relació **Brawler-Classe**:
  - Un Brawler pertany a una única Classe.
  - Una Classe pot tenir molts Brawlers.
- Relació **Brawler-Partida**:
  - Un Brawler pot estar jugant una Partida cada vegada.
  - Una Partida pot tenir molts Brawlers participants.
  - A més, un Brawler pot tenir moltes partides completades.
  - Una Partida pot haver estat completada per diversos brawlers.
  - Les relacions apunten a Partida (la classe abstracta): un Brawler pot jugar o haver completat partides de qualsevol tipus.
  - Assegura't de controlar mitjançant els setters que no pot haver-hi una partida activa al Brawler i a la vegada completada, així com que un Brawler amb partida activa no està a la llista de completadors de la partida.

A la darrera pàgina trobaràs el detall de les relacions i el model de dades a nivell UML, pot ajudar-te a aclarir-te.

Usa anotacions Hibernate per definir les entitats, l'herència, les relacions, claus primàries i restriccions (p. ex., nullable=false).

Inclou un fitxer de configuració hibernate.cfg.xml amb una unitat de persistència bàsica.

Pensa bé quan hauràs d'utilitzar fetch = FetchType.EAGER per poder obtenir tota la informació de la base de dades mitjançant el toString de Brawler. El toString ha de mostrar també el tipus de cada partida.

El nom de la base de dades serà **examenHibernate**.

Part 2: Implementació del patró DAO

Implementa el patró DAO per a l'entitat Brawler utilitzant Hibernate. Defineix una interfície IGenericDao amb un CRUD bàsic i IBrawlerDao amb 4 funcions addicionals que definirem a continuació. Després, implementa aquesta interfície en una classe BrawlerDao que heretarà de la classe GenericDao.

Requisits:

1. **Interfície IBrawlerDao:**
   - **Brawler[] torneig(int idBrawler1, int idBrawler2, int rondes);**
     - Simula un torneig entre dos brawlers durant el nombre de rondes indicat.

- A cada ronda, cada brawler perd tants punts de vida com el dany de l'habilitat de la classe del contrincant (la vida no pot baixar de zero).
    1. Qui acabi la ronda amb més vida guanya aquella ronda. Si empaten, la ronda no compta per a ningú.
- Si algun brawler mor (arriba a zero de vida) s'atura el torneig i el guanya el que queda viu. Si moren tots dos alhora, és empat.
- Si ningú mor, guanya el torneig qui hagi guanyat més rondes.
- El guanyador s'emporta la meitat dels trofeus de l'adversari. Utilitza intercanviTrofeus() per fer-ho.
- Controla els empats: no hi ha intercanvi de trofeus i s'indica per pantalla.
- Retorna un array amb els dos brawlers.

- **Brawler[] intercanviTrofeus(int idGuanyador, int idPerdedor, int trofeus);**
    - Transfereix la quantitat de trofeus indicada del brawler perdedor al guanyador.
    - Si el perdedor no té prou trofeus, la transferència falla, s'indica per pantalla i no s'aplica cap canvi.
    - Retorna un array amb els dos brawlers (primer el guanyador).
- **List<Brawler> repartirRecompensa(int trofeus);**
    - Reparteix trofeus a tots els brawlers multiplicant la quantitat indicada segons el seu rang:
        1. BRONZE x1
        2. PLATA x2
        3. OR x3
        4. DIAMANT x4
    - Gestiona el canvi de rang quan se superen els 100 trofeus: el brawler puja un rang i els trofeus es posen a 0. Si ja és DIAMANT, es queda com està.
    - Retorna la llista de tots els brawlers.
- **List<Brawler> completarPartides();**
    - Busca tots els brawlers que tenen una partida activa.
    - Per cadascun, aplica la lògica de completar partida:
        1. Esborra el brawler de la llista de participants de la partida i l'afegeix a la de completadors.
        2. La partida activa passa a null i s'afegeix a les partides completades del brawler.
        3. Si la partida és una PartidaRanked, rep la recompensa de trofeus. Si és una PartidaAmistosa, no rep trofeus.
        4. Si supera els 100 trofeus, puja de rang i els trofeus es posen a 0.

5. Si ja és DIAMANT i supera els 100 trofeus, en lloc de pujar de rang rep 5 punts de vida (màxim 10).
6. Si és DIAMANT i ja té 10 de vida, imprimeix "BRAWLER SUPREM" i es deixen tots els paràmetres com estan.

- Retorna la llista de tots els brawlers tractats.

2. **Fer DAO per a totes les classes té penalització d'1 punt.**

Avaluació:

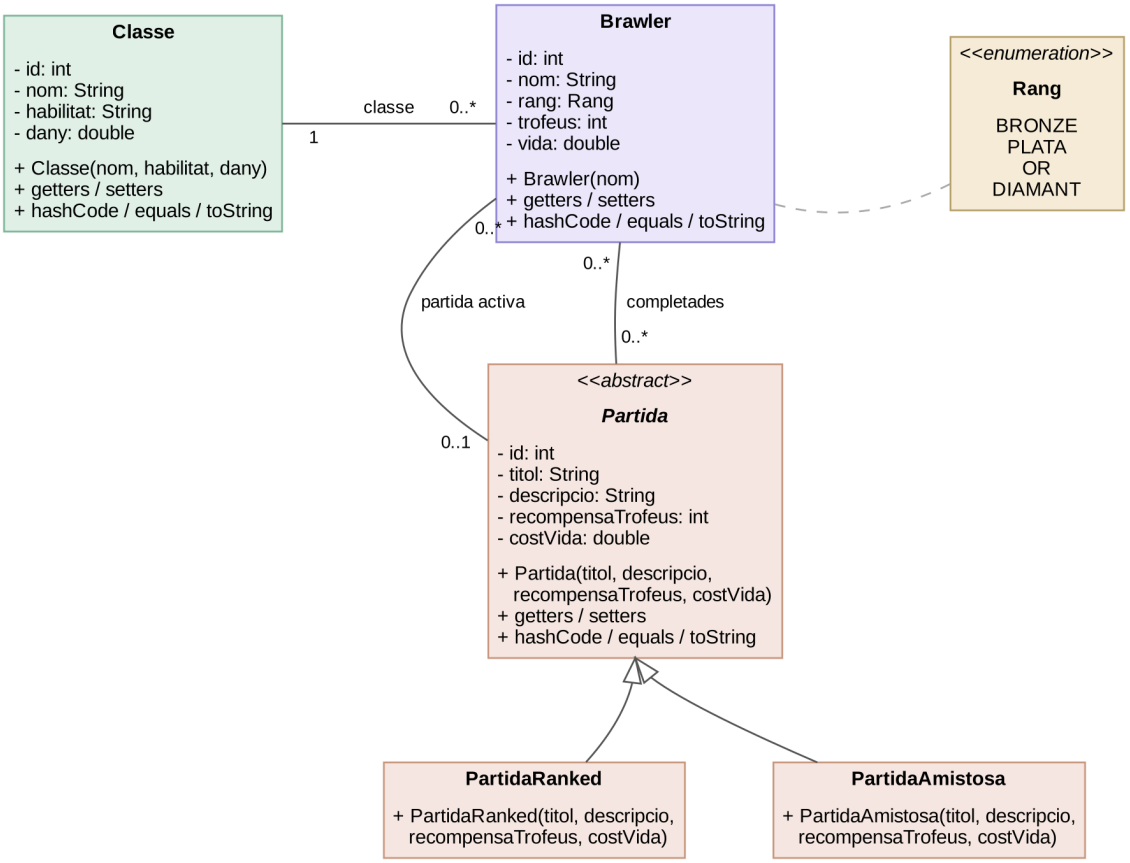
Part bàsica (45% de la nota):

1. Gestió d'excepcions, transaccions, commits i rollbacks. (**1** punt)
2. Classes correctes que compleixen tots els requisits del model. (**1** punt)
3. Herència correcta: Partida abstracta, SINGLE_TABLE amb columna discriminadora i subclasses sense atributs propis. (**0,5** punts)
4. Interfícies i classes DAO correctes segons els requisits i l'estructura proposada. (**1** punt)
5. Gestió dels toString() òptima per mostrar tota la informació (tipus de partida inclòs) només des de Brawler i sense caure en recursivitats. (**0,5** punts)
6. Correcta gestió del cost de vida quan s'assigna una partida activa al brawler. (**0,5** punts)

Part avançada (55% de la nota):

7. La funció intercanviTrofeus() valida correctament els trofeus del perdedor i aplica la transferència. (**0,5** punts)
8. La funció intercanviTrofeus() retorna l'array consultant a la base de dades després de persistir els canvis. (**0,5** punts)
9. La funció torneig() aplica correctament la lògica de rondes i dany per ronda i utilitza intercanviTrofeus(). (**0,5** punts)
10. La funció torneig() gestiona correctament l'aturada anticipada per mort i els empats, i determina el guanyador del torneig. (**0,5** punts)

11. La funció torneig() retorna l'array consultant a la base de dades després de persistir els canvis. (**0,5** punts)
12. La funció completarPartides() realitza correctament la consulta per obtenir els brawlers amb partida activa. (**0,5** punts)
13. La funció completarPartides() gestiona correctament la lògica de completar per a cadascun (participants, completadors, partida activa i completades). (**0,5** punts)
14. La funció completarPartides() diferencia PartidaRanked i PartidaAmistosa i gestiona correctament els requisits respecte als trofeus, rangs i vida. (**0,5** punts)
15. La funció completarPartides() retorna la llista consultant a la base de dades després de persistir els canvis. (**0,5** punts)
16. La funció repartirRecompensa() aplica correctament el multiplicador de rang a cada brawler. (**0,25** punts)
17. La funció repartirRecompensa() gestiona correctament el canvi de rang quan se superen els 100 trofeus. (**0,25** punts)
18. La funció repartirRecompensa() retorna la llista consultant a la base de dades després de persistir els canvis. (**0,5** punts)


Classe
- id: int
- nom: String
- habilitat: String
- dany: double
+ Classe(nom, habilitat, dany)
+ getters / setters
+ hashCode / equals / toString
classe
0..*
1
Brawler
- id: int
- nom: String
- rang: Rang
- trofeus: int
- vida: double
+ Brawler(nom)
+ getters / setters
+ hashCode / equals / toString
<<enumeration>>
Rang
BRONZE
PLATA
OR
DIAMANT
0..*
partida activa
0..1
0..*
completades
0..*
<<abstract>>
Partida
- id: int
- titol: String
- descripcio: String
- recompensaTrofeus: int
- costVida: double
+ Partida(titol, descripcio, recompensaTrofeus, costVida)
+ getters / setters
+ hashCode / equals / toString
PartidaRanked
+ PartidaRanked(titol, descripcio, recompensaTrofeus, costVida)
PartidaAmistosa
+ PartidaAmistosa(titol, descripcio, recompensaTrofeus, costVida)